PROJECT SYNOPSIS & TECHNICAL SPECIFICATION

BHARAT YATRA (■■■■ ■■■■■■)

National India Tourist Cities Travel Cost Calculator & AI Planning Studio

Domain: Domestic Indian Tourism & Budgeting	**Stack:** Python 3.11+ / Streamlit / SQLite / Gemini	**Deployment:** Docker / Render Cloud

1. Executive Summary & Problem Statement

1.1 Background & Transformation: The project originated as a Western corporate/grant tool based on United States General Services Administration (GSA) per-diem rates (FY24 allowances, Chicago travel policies, USD currency). It was radically overhauled and engineered from the ground up for **National Indian Tourism**.

1.2 Problem Addressed: Indian domestic travelers (families, solo tourists, backpackers, students) face significant fragmentation when planning holidays. Airfare, rail tariffs (IRCTC Vande Bharat vs. 3AC vs. Sleeper), hotel tiers, local commute (autos, cabs, metros), and monument entry fees are scattered across disparate portals. **Bharat Yatra** bridges this gap by unifying cost estimation in **Indian Rupee (■ INR)**, verified regional tourism data, dynamic itinerary curation, and personal trip tracking in one sleek platform.

2. Key System Capabilities

- **45+ Curated Indian Tourist Destinations:** Comprehensive data across North, South, East, West, Central, North-East, and Islands.
- **Realistic Multi-Tier Indian Cost Engine:** Automatic rate baselines for Budget (Hostels/Dhabas), Standard (3-Star Hotels/Family Cafes), and Luxury (Heritage Resorts/Fine Dining).
- **Inter-city Transit Estimation:** Pre-computed distance matrix connecting major departure hubs (Delhi, Mumbai, Bengaluru, Kolkata, Chennai, etc.) with accurate return fares.
- **Unified AI Travel Studio:** Fast dual-speed generation featuring *Instant Offline Mode (0.1s)* and *Live Gemini AI (gemini-3.8-flash)* for day-by-day itineraries and dynamic travel advice.
- **Context-Aware YatraAI Copilot:** Real-time interactive chatbot that queries the active user trip session (departure, destination, duration, budget, food, safety) to answer context-specific questions.
- **Full User Authentication & History:** SQLite-powered account creation, salted password hashing, 6-digit email OTP password recovery via Resend API, and a personal Saved Trips Dashboard.

3. Technology Stack Specification

Category	Technology / Package	Version	Functional Role
Core Runtime	Python	3.11 / 3.14	Application runtime, data manipulation, algorithm execution
Web Application	Streamlit	1.65.0	Reactive user interface, state management, interactive widgets
Data Engine	Pandas	3.0.6	CSV ingestion, dataset querying, route matrix lookups
Generative AI	Google GenAI SDK	2.21.0	Gemini 3.8 Flash model invocation for live itinerary generation
Transactional Email	Resend SDK	2.36.0	Direct email delivery of 6-digit password reset verification OTPs
Database	SQLite3	Built-in	Relational storage for user accounts, OTP records, and saved trips
Security / Crypto	Python hashlib	Built-in	SHA-256 cryptographic password hashing with custom salt
Containerization	Docker	3.11-slim	Containerized runtime with dynamic $PORT binding for Render
Automated Testing	Pytest	9.1.1	Automated unit test suite validating formulas, data, and fallbacks

4. Architecture & File Breakdown

The project follows a modular, decoupled architecture separating UI presentation, computation, authentication, data storage, and external API gateways:

```
travel_cost_calculator/
■■■ .streamlit/
■   ■■■ config.toml            # Light nature theme (#2D6A4F), headless config, minimal toolbar
■   ■■■ secrets.toml           # Encrypted local API keys (Gemini & Resend) — git-ignored
■■■ data/
■   ■■■ bharat_yatra.db        # SQLite relational database (Users, OTPs, Saved Trips)
■   ■■■ india_tourist_cities.csv# Database of 45+ cities (rates, attractions, cuisine, seasons)
■   ■■■ india_routes.csv       # Hub-to-destination transit distance and return fare baselines
■■■ src/
■   ■■■ app.py                 # Multi-page router, Dashboard, Calculator, Auth & Nature CSS
■   ■■■ auth.py                # Database connection, salted hashing, OTPs, trip persistence
■   ■■■ travel_calculator.py   # Cost math, route lookups, INR formatter, summary generator
■   ■■■ ai_planner.py          # Gemini 3.8 Flash caller, fast single-pass planner, Copilot
■   ■■■ data.py                # Static definitions (departure hubs, travel styles, transit modes)
■■■ tests/
■   ■■■ test_calculator.py     # 9 pytest test cases verifying computation and data structure
■■■ app.py                     # Root execution wrapper for Streamlit runner
■■■ Dockerfile                 # Production Docker image with dynamic $PORT binding
■■■ requirements.txt           # Dependency declaration list
■■■ README.md                  # Comprehensive documentation and deployment guide
```

5. Detailed Module Logic & Algorithms

5.1 Mathematical Budget Calculation Engine (travel_calculator.py)

The cost model calculates realistic domestic budgets based on 5 independent expense vectors plus an emergency contingency margin:

• **Duration & Lodging:** Total Days = (End Date - Start Date) + 1. Hotel Nights = max(1, Total Days - 1).
• **Inter-city Transit:** Flat cost per vehicle for Personal Drive / Cab; Per-person Return Fare × Travelers for Flight, Train, and Bus.
• **Lodging Cost:** Hotel Nights × Rooms Needed × Nightly Hotel Rate.
• **Meals & Dining:** Total Days × Travelers × Food Rate per Day.
• **Local Commute:** Total Days × Daily Transit Rate.
• **Sightseeing & Monument Passes:** Total Days × Travelers × Daily Ticket Baseline.
• **Contingency Buffer:** Subtotal × (Buffer % / 100). Grand Total = Subtotal + Buffer.
• **Currency Formatting:** The proprietary format_inr() function splits digits according to the Indian numbering system (grouping the final 3 digits, then pairs of 2 digits, e.g. ■1,25,000).

5.2 Dual-Engine AI Architecture (ai_planner.py)

To avoid the latency of multi-call conversational loops, the AI subsystem was optimized into a **Single-Pass Fast Generation** model:

1. **Single Prompt Synthesis:** The prompt requests both the Day-by-Day schedule and 4 specific money-saving hacks simultaneously, reducing round-trip API latency by over 65%.
2. **Dynamic Model Selection:** Uses Google's gemini-3.8-flash with a 1200-token envelope for sub-3-second responses.
3. **Dual Speed Modes:** Users can select ■ *Instant Mode (0.1s)* which compiles structured itineraries instantly from the local destination database, or ■ *Live Gemini AI* for customized LLM generation.
4. **Dynamic Context-Aware Copilot:** The YatraAI assistant queries the active trip context dictionary. When the user asks about their selected trip, departure hub, budget, or foods, the Copilot dynamically answers using their exact session parameters instead of repeating static text.

5.3 Authentication & Resend OTP Gateway (auth.py)

• **Password Security:** Uses SHA-256 with an application-specific cryptographic salt to prevent rainbow table attacks.
• **Resend Email OTP Dispatch:** When a password reset is initiated, a cryptographically random 6-digit numeric OTP is stored in the otps table with a 15-minute expiration timestamp and dispatched via Resend API from onboarding@resend.dev.
• **Trip History Persistence:** Provides CRUD operations for users to save calculated trips to their account and review them on the Dashboard.

6. Database Schema & Data Models

The application utilizes SQLite3 for local persistence and supports PostgreSQL in production via connection strings:

Table Name	Column	Data Type	Description & Constraints
users	id name email password_hash created_at	INTEGER PK TEXT TEXT UNIQUE TEXT TIMESTAMP	User account registry. Unique indexed email. Salted SHA-256 password hash.
otps	email otp_code created_at expires_at	TEXT PK TEXT TIMESTAMP TIMESTAMP	Password reset verification codes. 6-digit OTP with 15-minute TTL.
saved_trips	id user_email destination origin travel_dates travelers travel_style total_budget cost_per_person created_at	INTEGER PK TEXT TEXT TEXT TEXT INTEGER TEXT TEXT TEXT TIMESTAMP	Saved trip estimates. Foreign reference to user email. Stores complete trip snapshot for dashboard display.

7. Deployment & DevOps Lifecycle

7.1 Docker Containerization: The application is containerized using an official lightweight python:3.11-slim base image. It incorporates a non-root environment, dynamic $PORT resolution (--server.port=${PORT:-8501}) allowing immediate deployment to Render, Railway, AWS, or GCP without port conflicts.

7.2 Cloud Deployment on Render.com:
1. **Continuous Integration:** Connected directly to GitHub repository ShaikhVaiz/travel_cost_calculator.
2. **Automatic Builds:** Every git push automatically triggers a Docker build and zero-downtime rolling deployment.
3. **Environment Secrets:** Configured with production environment variables GEMINI_API_KEY and RESEND_API_KEY.
4. **Public Availability:** Globally accessible via HTTPS at https://bharat-yatra.onrender.com.

7.3 Quality Assurance & Verification: Pytest executes 9 automated test suites verifying dataset integrity, state/city filtering, inter-city fare estimations, INR currency formatting, budget arithmetic, and AI offline fallback safety.